From Coloured Points to Printable Textured Meshes

Anas Khan MBZUAI anas.khan@mbzuai.ac.ae CV802 Sub-project 2: Surface Extraction



Figure 1. Best mesh per method on VGGSfM: *textured* and *shaded*. Red: triangles seen from the back.

		VGGSfM (main input, 139k pts)				MVS (487k pts)		MVS-clean		
Method	Faces	CD %↓	F@1%↑	F@0.5%↑	NC↑	CD %↓	F@1%↑	CD %↓	F@1%↑	WT
Poisson (ours)	1.02M	**0.149**	**0.999**	**0.988**	0.75	0.127	0.999	0.127	0.999	
IER	17k	0.187	0.998	0.975	0.82	0.120	**1.000**	0.123	**1.000**	
Point2Mesh	10k	0.553	0.844	0.530	0.80	3.977	0.133	1.298	0.387	✓
MeshAnything V2	1.4k	0.295	0.987	0.858	0.82	0.518	0.873	0.438	0.908	
DHSP	6.0k	0.583	0.901	0.407	**0.89**	0.636	0.900	0.627	0.916	✓
NKSR*	2.09M	0.156	0.998	0.973	0.75	**0.080**	**1.000**	**0.080**	**1.000**	
SAP*	64k	0.195	0.998	0.961	0.81	0.197	0.963	0.239	0.948	✓
2DGS*	657k	0.289	0.972	0.864	0.75	0.308	0.938	0.336	0.928	

Table 1. Best run per method vs. its own input dots. CD: mean mesh–dot gap (Chamfer), % of the scan's diagonal, lower is better; F@τ: share within τ of each other, higher is better; NC: agreement of surface directions; WT: closed, printable. * Beyond the course list.

1 What we did

Sub-project 1 gave us coloured dots on a person in a light shirt. We turn them into textured 3D meshes (`.obj`) in eight ways, on three versions of the scan: VGGSfM (139k dots, cleaned with the person's outline in the photos; main input), the denser COLMAP MVS (487k), and MVS with 404 stray floating dots removed (MVS-clean). A shared last step adds texture, scores and previews, and our web viewer (`run_viewer.sh`, no GPU) shows them side by side.

2 How

Our own Poisson (no library). Each dot gets an outward arrow from its 24 nearest neighbours, made consistent across the body. The arrows $\mathbf{V}$ define an inside-or-outside function χ on a 384^3 grid whose slope matches them ($\Delta\chi = \nabla\cdot\mathbf{V}$), plus a pull that keeps the surface on the dots. Its boundary becomes triangles, and parts far from any dot are trimmed: 1.0 M triangles in 39 s.

Making the course methods work on a real scan. **IER** gets 12.5k evenly spread dots, the density it was trained on. **MeshAnything V2** learned from clean meshes: raw dots give F@1% 0.51, our Poisson mesh via its authors' preparation step 0.987. **Point2Mesh** was misled by the direction of our noisy arrows; the paper's option to ignore it lifts F@1% from 0.65 to 0.84. **DHSP** first fits the dots but crumples the surface (NC 0.65), and its next step (flattening the surface into an image) stalled for 15 h. Rebuilding a clean, simpler surface first (3k/6k triangles) lets it finish in seconds: smooth (NC 0.89), with its own colours.

Three more methods. NKSR (a learned field built for real scans), Shape As Points (Poisson made trainable) and 2D Gaussian Splatting (learned from the 125 photos). With no true 3D model, Table 1 scores each mesh against its own input dots.

3 What we learned

(1) Classic code holds up: our Poisson ties the best learned methods on VGGSfM; NKSR wins on the denser MVS. **(2) Close to the dots is not a good surface:** DHSP's first stage scores F@1% 0.999 yet is crumpled, and IER's triangles disagree about which side is out (red in Fig. 1). **(3) The input matters most:** MeshAnything needs mesh-like input; Point2Mesh falls apart on MVS with its stray dots (F@1% 0.13) and partly recovers without them (0.39). **(4) Closed meshes print:** SAP, DHSP, Point2Mesh and our untrimmed Poisson are watertight, exported as 180 mm STL files.